



Name: Shibaji Biswas

UID: 24BAI71018

Branch: CSE – AIML (CS221)

Section/Group: 24AIT-NTTP-3[B]

Semester: 5th

Date of Performance: 08/07/2026

Subject: Full Stack - II

Subject Code: 24CSP-337

EXPERIMENT – 1.1

1. AIM: To design and develop a dynamic post composer interface supporting multiple platforms with constraint validation.

2. OBJECTIVE:

1. To understand multi-platform content handling.
2. To implement real-time validation mechanisms.
3. To design responsive and user-friendly UI components.

3. SOFTWARE REQUIREMENTS:

1. Windows/Linux/macOS
2. Node.js (LTS Version)
3. NPM (Node Package Manager)
4. Visual Studio Code
5. Google Chrome / Microsoft Edge
6. React.js
7. Command Prompt / Terminal

4. THEORY

React is an open-source JavaScript library developed by Meta (Facebook) for building fast, dynamic, and interactive user interfaces. It follows a component-based architecture, where the user interface is divided into reusable components. This approach improves code reusability, readability, and maintainability.

In modern social media applications, users often create content for multiple platforms such as Twitter, LinkedIn, and Instagram. Each platform has different character limits and posting rules. Therefore, a Post Composer is designed to help users create content while validating platform-specific constraints

before publishing. Real-time validation improves user experience by immediately informing users if the entered content exceeds the allowed limit.

The Post Composer developed in this experiment is implemented using React Functional Components and React Hooks. The `useState()` hook is used to store and manage dynamic data such as the selected platform and the content entered by the user. Whenever the state changes, React automatically updates the user interface without reloading the entire webpage.

User interactions such as typing in the textarea or selecting a platform are handled using event handling (`onChange`). The application also uses controlled components, where the value of form elements is managed through React state. This ensures synchronization between the user interface and the application's data.

The application dynamically calculates the number of characters entered using the `length` property and compares it with the predefined character limit of the selected platform. Based on this comparison, conditional rendering is used to display appropriate validation messages such as "Ready to Post" or "Character Limit Exceeded."

Overall, this experiment demonstrates the practical implementation of React concepts including components, JSX, state management, event handling, and controlled components,

5. IMPLEMENTATION/PROCEDURE

1. Install Node.js and verify the installation using `node -v` and `npm -v`.
2. Create a React project using `npx create-react-app post-composer`.
3. Open the project in Visual Studio Code and run it using `npm start`.
4. Design the Post Composer interface with a heading, platform dropdown, and textarea.
5. Implement React State (`useState`) to manage the selected platform and post content.
6. Add a live character counter using the `length` property.
7. Define platform-specific character limits (Twitter, LinkedIn, and Instagram).
8. Implement real-time validation to display appropriate messages based on the selected platform's character limit.
9. Test the application by entering different post lengths and switching between platforms.
10. Verify that the application displays the correct character count and validation message for each platform.

6. CODE

```
import { useState } from "react";

function App() {
  const [platform, setPlatform] = useState("Twitter");
  const [post, setPost] = useState("");
```

```
const limits = {
  Twitter: 280,
  LinkedIn: 3000,
  Instagram: 2200,
};

const limit = limits[platform];
const isExceeded = post.length > limit;

return (
  <div
    style={{
      backgroundColor: "#f4f6f9",
      minHeight: "100vh",
      padding: "40px",
      fontFamily: "Arial, sans-serif",
    }}
  >
    <div
      style={{
        width: "650px",
        margin: "auto",
        backgroundColor: "#ffffff",
        padding: "30px",
        borderRadius: "15px",
        boxShadow: "0px 4px 15px rgba(0,0,0,0.2)",
      }}
    >
      <h1
        style={{
          textAlign: "center",
          color: "#2c3e50",
          marginBottom: "30px",
        }}
      >
        Social Media Post Composer
```

```
</h1>
```

```
<label>
```

```
  <b>Select Platform</b>
```

```
</label>
```

```
<br />
```

```
<br />
```

```
<select
```

```
  value={platform}
```

```
  onChange={(e) => setPlatform(e.target.value)}
```

```
  style={{
```

```
    width: "100%",
```

```
    padding: "10px",
```

```
    borderRadius: "8px",
```

```
    fontSize: "16px",
```

```
  }}>
```

```
  <option>Twitter</option>
```

```
  <option>LinkedIn</option>
```

```
  <option>Instagram</option>
```

```
</select>
```

```
<br />
```

```
<br />
```

```
<label>
```

```
  <b>Write Your Post</b>
```

```
</label>
```

```
<br />
```

```
<br />
```

```
<textarea
```

```
  rows="6"
```

```
  placeholder="Type your post here..."
```

```
value={post}
onChange={(e) => setPost(e.target.value)}
style={{
  width: "100%",
  padding: "10px",
  borderRadius: "10px",
  border: "2px solid #ccc",
  fontSize: "16px",
  resize: "none",
}}
</textarea>
```

```
<br />
```

```
<br />
```

```
<p
  style={{
    fontWeight: "bold",
    color: isExceeded ? "red" : "#2c3e50",
  }}
>
  Characters: {post.length} / {limit}
</p>
```

```
{isExceeded ? (
  <p
    style={{
      color: "red",
      fontWeight: "bold",
    }}
  >
    Character Limit Exceeded!
  </p>
```

```
) : (
  <p
    style={{
      color: "green",
```

```

        fontWeight: "bold",
      }}
    >
      Ready to Post
    </p>
  )}

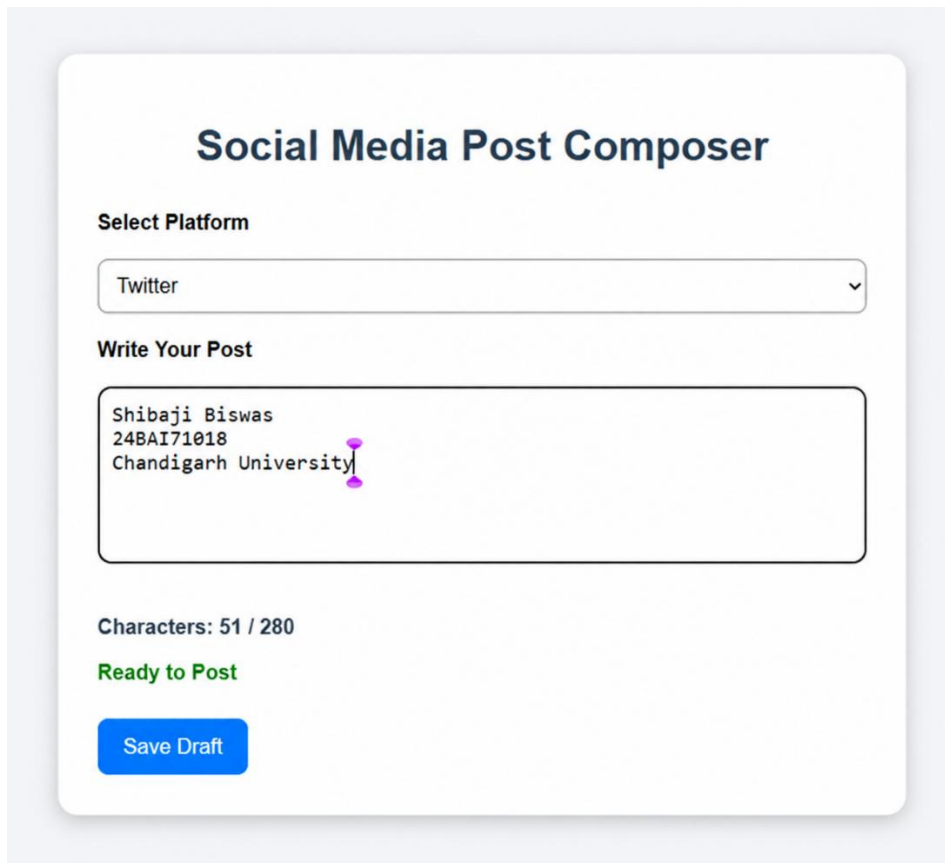
  <button
    style={{
      backgroundColor: "#007bff",
      color: "white",
      border: "none",
      padding: "12px 20px",
      borderRadius: "8px",
      fontSize: "16px",
      cursor: "pointer",
      marginTop: "10px",
    }}
  >
    Save Draft
  </button>
</div>
</div>
);
}

export default App;

```

7. OUTPUT

The Social Media Post Composer is a user-friendly interface that allows users to create and manage social media posts according to the selected platform's constraints. Users can select a platform such as Twitter, enter their post content in the text area, and view the real-time character count, which is compared with the platform's maximum character limit. The system displays a status message such as “Ready to Post” when the content is within the allowed limit and provides a “Save Draft” button to save the written content for future use. This interface demonstrates the use of platform-specific constraints, interactive form elements, real-time validation, and dynamic user feedback in a React-based application.

A mockup of a social media post composer interface. At the top, the title "Social Media Post Composer" is centered in a bold, dark blue font. Below the title, there is a section labeled "Select Platform" with a dropdown menu currently showing "Twitter". Underneath, the "Write Your Post" section features a text input area containing the text "Shibaji Biswas", "24BAI71018", and "Chandigarh University". To the right of the text, there are two small purple dots. Below the text input, the character count "Characters: 51 / 280" is displayed. A green status message "Ready to Post" is shown below the character count. At the bottom, there is a blue button labeled "Save Draft".

Social Media Post Composer

Select Platform

Twitter

Write Your Post

Shibaji Biswas
24BAI71018
Chandigarh University

Characters: 51 / 280

Ready to Post

Save Draft

Fig 1.1: Post Composer UI with Platform Constraints

8. LEARNING OUTCOME

- Understand the basic concepts and purpose of a social media post composer interface.
- Design and develop a user-friendly post composer UI using React.js.
- Implement platform selection features for platforms such as Twitter, LinkedIn, and Instagram.
- Apply platform-specific character limits to ensure content complies with each platform's constraints.
- Use React state management to handle user input and dynamic UI updates.
- Display real-time character counts and warning messages when the content exceeds the allowed limit.
- Improve user experience through responsive and interactive interface components.
- Develop practical skills in building reusable, constraint-based frontend applications.

Experiment: 1.2

Aim: To implement a draft management system that allows users to save, retrieve, and manage post drafts within the frontend, with optional simulation of backend interactions.

Objective:

- To manage draft data using React state.
- To implement Create, Read, Update, and Delete (CRUD) operations for drafts.
- To understand asynchronous workflows in frontend applications.
- To simulate backend communication using mock APIs or local storage.
- To enhance user experience through efficient draft handling.

Software Requirement:

- Operating System: Windows 10/11 or Linux
- Programming Software/IDE: Visual Studio Code /MySQL Workbench
- Processor: Intel Core i3 or higher
- RAM: Minimum 4 GB

Theory:

A Draft Management System allows users to temporarily save content before publishing it. In React.js, this functionality is implemented using state management techniques such as `useState`, where draft data is stored and updated dynamically. Users can perform CRUD (Create, Read, Update, Delete) operations on drafts without requiring an actual backend server. For improved user experience, drafts may also be stored in the browser using `localStorage` or simulated with mock APIs to imitate asynchronous server communication. This approach helps developers build and test frontend features independently before integrating real backend services.

Implementation :

1. Created a React application using Create React App.
2. Imported the `useState` Hook for state management.
3. Created state variables for platform, post, drafts, image, and editing mode.
4. Designed the user interface with a dropdown, textarea, image upload, and action buttons.

5. Implemented Save Draft functionality to store posts dynamically.
6. Added Edit Draft functionality to modify existing drafts.
7. Implemented Delete Draft functionality to remove unwanted drafts.

Code:

```
import
React, { useState } from "react";

import
"./App.css";

function
App() {
  const [platform, setPlatform] =
useState("Instagram");

  const [post, setPost] =
useState("");

  const [drafts, setDrafts] = useState([]);

  const [image, setImage] = useState(null);

  const [editing, setEditing] = useState(null);

  const saveDraft = () => {
    if (!post.trim()) {
      alert("Write something
first!");
      return;
    }

    const draft = { text: post, platform, image
};

    if (editing !== null) {
      const copy = [...drafts];
```

```
    copy[editing] = draft;

    setDrafts(copy);

    setEditing(null);

  } else {

    setDrafts([draft, ...drafts]);

  }

  setPost("");

  setImage(null);

};

const deleteDraft = (index) =>

setDrafts(drafts.filter((_, i) => i !==

index));

const editDraft = (index) => {

  setPost(drafts[index].text);

  setPlatform(drafts[index].platform);

  setImage(drafts[index].image);

  setEditing(index);

};

const publish = () => {

  alert("🎉 Post Published

Successfully!");

  setPost("");

  setImage(null);

};

const uploadImage = (e) => {

  if (e.target.files[0])
```

```
setImage(URL.createObjectURL(e.target.files[0]));
```

```
};
```

```
return (
```

```
<div className="app">
```

```
<div className="card">
```

```
<h1> 🚀 Social Media Composer</h1>
```

```
<select
```

```
value={platform}
```

```
onChange={(e) =>
```

```
setPlatform(e.target.value)}
```

```
>
```

```
<option>Instagram</option>
```

```
<option>Facebook</option>
```

```
<option>LinkedIn</option>
```

```
<option>X
```

```
(Twitter)</option>
```

```
</select>
```

```
<textarea
```

```
placeholder="What's on your
```

```
mind?"
```

```
value={post}
```

```
onChange={(e) =>
```

```
setPost(e.target.value)}
```

```
/>
```

```
{image} && <img src={image}
```

```
alt="" className="preview" />}
```

```
<input type="file"
```

```
onChange={uploadImage} />

<button onClick={saveDraft}>

  {editing !== null ? "Update

Draft" : "Save Draft"}

</button>

<button
onClick={publish}>Publish</button>

</div>

<div
className="draftSection">

  <h2>Saved Drafts</h2>

  {drafts.map((item, index) => (

    <div key={index}>

      <h3>{item.platform}</h3>

      <p>{item.text}</p>

      {item.image && (

        <img src={item.image}

alt="" width="150" />

      )}

      <button onClick={() =>

editDraft(index)}>Edit</button>

      <button onClick={() =>

deleteDraft(index)}>Delete</button>

    </div>

  )}

</div>
```

```
</div>
```

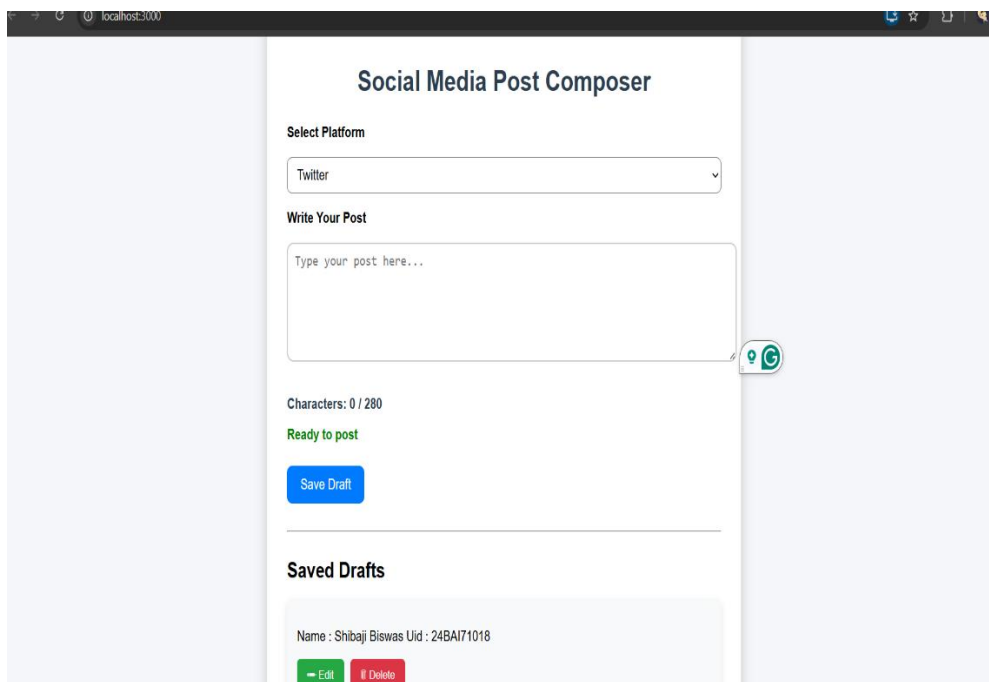
```
);
```

```
}
```

```
export
```

```
default App;
```

Output:



Learning Outcomes :

After completing this experiment, I was able to:

1. Understand React state management using useState.
2. Implement CRUD operations in a frontend application.
3. Develop a functional draft management system.
4. Handle user interactions efficiently in React. Simulate backend-like behavior using frontend techniques. Improve problem-solving and debugging skills while building interactive user interfaces.